

APUNTES JAVA

POO · Arrays · Excepciones

1. CLASES Y CONSTRUCTORES

1.1 Clase básica con un constructor

Una clase es una plantilla. El constructor se llama igual que la clase, no tiene tipo de retorno y se ejecuta al crear el objeto con new.

```
public class Persona {
    // Atributos SIEMPRE privados (encapsulamiento)
    private String nombre;
    private int edad;

    // Constructor
    public Persona(String nombre, int edad) {
        this.nombre = nombre;    // this -> atributo del objeto
        this.edad = edad;
    }

    // Getters
    public String getNombre() { return nombre; }
    public int    getEdad()    { return edad; }

    // Setters
    public void setEdad(int edad) {
        if (edad >= 0) this.edad = edad;
    }

    // toString (se llama al hacer System.out.println(objeto))
    @Override
    public String toString() {
        return "Persona[" + nombre + ", " + edad + "]";
    }
}
```

1.2 Varios constructores (sobrecarga)

☐ Puedes tener varios constructores con distinto número o tipo de parámetros.

```
public class Rectangulo {
    private double ancho;
    private double alto;

    // Constructor 1: sin parámetros (por defecto)
    public Rectangulo() {
        this.ancho = 1.0;
        this.alto  = 1.0;
    }

    // Constructor 2: cuadrado
    public Rectangulo(double lado) {
        this.ancho = lado;
        this.alto  = lado;
    }

    // Constructor 3: completo
    public Rectangulo(double ancho, double alto) {
        this.ancho = ancho;
        this.alto  = alto;
    }

    public double area() { return ancho * alto; }
}
```

1.3 Llamar a un constructor desde otro con this()

```
public class Circulo {
    private double radio;
    private String color;

    public Circulo() {
        this(1.0, "rojo");    // llama al constructor de abajo
    }

    public Circulo(double radio, String color) {
        this.radio = radio;
        this.color = color;
    }
}
```

1.4 Atributo static (compartido por todos los objetos)

□ static → pertenece a la clase, no al objeto. Todos los objetos comparten el mismo valor.

```
public class Cuenta {
    private static int totalCuentas = 0;    // compartido
    private int numeroCuenta;
    private double saldo;

    public Cuenta(double saldoInicial) {
        totalCuentas++;                    // se incrementa en cada new
        this.numeroCuenta = totalCuentas;
        this.saldo = saldoInicial;
    }

    public static int getTotalCuentas() { return totalCuentas; }
}

// En main:
Cuenta c1 = new Cuenta(100);
Cuenta c2 = new Cuenta(200);
System.out.println(Cuenta.getTotalCuentas()); // 2
```

1.5 Referencia vs copia de objeto

□ Cuenta c2 = c1; → NO copia el objeto, ambas variables apuntan al MISMO objeto.

```
Cuenta c1 = new Cuenta(100);
Cuenta c2 = c1;           // c2 y c1 apuntan al MISMO objeto
c2.setSaldo(999);
System.out.println(c1.getSaldo()); // 999 (mismo objeto!)

// Para copiar un array:
int[] a = {1, 2, 3};
int[] b = a;               // ALIAS (misma referencia)
int[] c = a.clone();       // COPIA real (objetos independientes)
b[0] = 50;                 // modifica a también
c[1] = 100;                // NO modifica a
System.out.println(a[0]); // 50 (b sí lo modifica)
System.out.println(a[1]); // 2  (c no lo modifica)
```

2. ARRAYS

2.1 Declaración y creación

```
// Forma 1: declarar y asignar valores directamente
int[] numeros = {10, 20, 30, 40, 50};

// Forma 2: declarar tamaño (los valores son 0 por defecto)
int[] notas = new int[5];
notas[0] = 8;
notas[1] = 7;

// Longitud del array
System.out.println(numeros.length); // 5

// Array de Strings
String[] nombres = {"Ana", "Luis", "Marta"};

// Array de objetos (ojo: referencias a null hasta inicializarlas)
Persona[] personas = new Persona[3]; // 3 referencias NULL
personas[0] = new Persona("Ana", 20); // ahora sí es un objeto
```

2.2 Recorrer con for tradicional

```
int[] nums = {3, 7, 1, 9, 4};

for (int i = 0; i < nums.length; i++) {
    System.out.println("Posición " + i + ": " + nums[i]);
}

// Suma y media
double suma = 0;
for (int i = 0; i < nums.length; i++) {
    suma += nums[i];
}
double media = suma / nums.length;
System.out.println("Media: " + media);
```

2.3 Recorrer con for-each (foreach)

☐ El foreach es más cómodo pero NO te da el índice y NO puedes modificar el array.

```
int[] nums = {3, 7, 1, 9, 4};

for (int n : nums) {
    System.out.println(n);
}

// Con objetos:
Persona[] personas = { new Persona("Ana", 20), new Persona("Luis", 30) };
for (Persona p : personas) {
    System.out.println(p.getNombre());
}
```

2.4 Recorrer en orden INVERSO

```
int[] nums = {1, 2, 3, 4, 5};

for (int i = nums.length - 1; i >= 0; i--) {
    System.out.print(nums[i] + " ");
}
// Salida: 5 4 3 2 1
```

2.5 Invertir un array (crear array nuevo)

```
int[] original = {1, 2, 3, 4, 5};
int[] invertido = new int[original.length];

for (int i = 0; i < original.length; i++) {
    invertido[i] = original[original.length - 1 - i];
}

// Mostrar invertido
for (int n : invertido) {
    System.out.print(n + " ");
}
// Salida: 5 4 3 2 1
```

2.6 Copiar array manualmente (sin clone)

```
int[] origen = {10, 20, 30};
int[] destino = new int[origen.length];

for (int i = 0; i < origen.length; i++) {
    destino[i] = origen[i];
}
```

2.7 Buscar el mayor y el menor

```
int[] nums = {4, 7, 2, 9, 1};

int mayor = nums[0];
int menor = nums[0];

for (int i = 1; i < nums.length; i++) {
    if (nums[i] > mayor) mayor = nums[i];
    if (nums[i] < menor) menor = nums[i];
}
System.out.println("Mayor: " + mayor); // 9
System.out.println("Menor: " + menor); // 1
```

2.8 Buscar un elemento y su posición

```
int[] nums = {5, 3, 8, 1, 9};
int buscar = 8;
boolean encontrado = false;

for (int i = 0; i < nums.length; i++) {
    if (nums[i] == buscar) {
```

```

        System.out.println("Encontrado en posición: " + i);
        encontrado = true;
        break; // salir en cuanto lo encuentre
    }
}
if (!encontrado) System.out.println("No encontrado");

```

2.9 Contar pares, impares y positivos

```

int[] nums = {4, -3, 7, 0, -2, 8};
int pares = 0, impares = 0, positivos = 0;

for (int n : nums) {
    if (n % 2 == 0) pares++;
    else           impares++;
    if (n > 0)      positivos++;
}
System.out.println("Pares: " + pares);
System.out.println("Impares: " + impares);
System.out.println("Positivos: " + positivos);

```

2.10 Comparar arrays: equals vs Arrays.equals

❑ equals() en arrays compara REFERENCIAS (siempre false si son objetos distintos). Usa Arrays.equals() para comparar CONTENIDO.

```

import java.util.Arrays;

int[] a = {1, 2, 3};
int[] b = {1, 2, 3};

System.out.println(a.equals(b));           // false (compara referencias)
System.out.println(Arrays.equals(a,b));    // true  (compara contenido)

```

2.11 Arrays como parámetros en métodos

❑ Cuando pasas un array a un método, se pasa la REFERENCIA. El método SÍ puede modificar el original.

```

// Este método SÍ modifica el array original
public static void multiplicarPorDos(int[] array) {
    for (int i = 0; i < array.length; i++) {
        array[i] *= 2; // modifica directamente
    }
}

// Este método NO modifica el array (foreach + var local)
public static void intentarModificar(int[] array) {
    for (int n : array) {
        n *= 2; // n es copia local, el array NO cambia
    }
}

// En main:
int[] nums = {1, 2, 3};
multiplicarPorDos(nums);
System.out.println(Arrays.toString(nums)); // [2, 4, 6]

```

2.12 Arrays de objetos completo

```
public class Alumno {
    private String nombre;
    private double nota;

    public Alumno(String nombre, double nota) {
        this.nombre = nombre;
        this.nota = nota;
    }
    public String getNombre() { return nombre; }
    public double getNota() { return nota; }
    public void setNota(double n) { nota = n; }

    @Override
    public String toString() {
        return nombre + " -> " + nota;
    }
}

// En main:
Alumno[] alumnos = new Alumno[3];
alumnos[0] = new Alumno("Ana", 8.5);
alumnos[1] = new Alumno("Luis", 6.0);
alumnos[2] = new Alumno("Marta", 9.0);

// Mostrar todos
for (Alumno a : alumnos) {
    System.out.println(a);
}

// Encontrar el mejor
Alumno mejor = alumnos[0];
for (int i = 1; i < alumnos.length; i++) {
    if (alumnos[i].getNota() > mejor.getNota())
        mejor = alumnos[i];
}
System.out.println("Mejor: " + mejor);

// Subir nota a todos
for (Alumno a : alumnos) {
    a.setNota(a.getNota() + 1);
}
```

3. ARRAYLIST

□ ArrayList es como un array dinámico: crece y encoge. Necesita import java.util.ArrayList;

3.1 Crear y usar un ArrayList

```
import java.util.ArrayList;

ArrayList<String> lista = new ArrayList<>();

// Añadir
lista.add("Ana");
lista.add("Luis");
lista.add("Marta");

// Obtener elemento
String primero = lista.get(0); // "Ana"

// Tamaño
int tam = lista.size(); // 3

// Eliminar por índice
lista.remove(1); // elimina "Luis"

// Eliminar por valor
lista.remove("Marta");

// Comprobar si contiene
boolean existe = lista.contains("Ana"); // true

// Recorrer con for-each
for (String s : lista) {
    System.out.println(s);
}

// Recorrer con for con índice
for (int i = 0; i < lista.size(); i++) {
    System.out.println(lista.get(i));
}
```

3.2 ArrayList de objetos

```
ArrayList<Alumno> alumnos = new ArrayList<>();
alumnos.add(new Alumno("Ana", 8.5));
alumnos.add(new Alumno("Luis", 6.0));

for (Alumno a : alumnos) {
    System.out.println(a.getNombre() + " " + a.getNota());
}
```


4. EXCEPCIONES

4.1 Try-catch básico

- ❑ Si una excepción no se captura se propaga hacia arriba hasta main. Si llega a main sin capturarse, termina el programa.

```
try {
    int resultado = 10 / 0; // lanza ArithmeticException
    System.out.println(resultado);
} catch (ArithmeticException e) {
    System.out.println("Error: " + e.getMessage());
}
System.out.println("El programa continúa...");
```

4.2 Múltiples catch

- ❑ Puedes tener tantos catch como necesites. Se ejecuta solo el primero que coincide.

```
try {
    String s = null;
    int[] arr = new int[5];

    s.length(); // NullPointerException
    arr[10] = 1; // ArrayIndexOutOfBoundsException
} catch (NullPointerException e) {
    System.out.println("Referencia nula: " + e.getMessage());
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Índice fuera de rango: " + e.getMessage());
} catch (Exception e) {
    // Captura cualquier otra excepción (debe ir AL FINAL)
    System.out.println("Error genérico: " + e.getMessage());
}
```

4.3 Try-catch-finally

- ❑ finally se ejecuta SIEMPRE, haya excepción o no. Se usa para cerrar recursos.

```
try {
    int[] arr = new int[3];
    arr[5] = 1; // excepción
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Índice inválido");
} finally {
    System.out.println("Esto se ejecuta siempre");
}
```

4.4 Lanzar excepción con throw

```
public static int dividir(int a, int b) {
    if (b == 0) {
        throw new ArithmeticException("No se puede dividir por cero");
    }
    return a / b;
}
```

```
// En main:
try {
    int res = dividir(10, 0);
} catch (ArithmeticException e) {
    System.out.println(e.getMessage());
}
```

4.5 Declarar excepción en la firma con throws

☐ throws avisa al que llama al método que puede lanzar esa excepción y DEBE manejarla.

```
public static void retirar(double cantidad) throws Exception {
    if (cantidad > saldo) {
        throw new Exception("Saldo insuficiente");
    }
    saldo -= cantidad;
}

// El que llama DEBE usar try-catch (o propagar con throws):
try {
    retirar(500);
} catch (Exception e) {
    System.out.println(e.getMessage());
}
```

4.6 Excepción personalizada (checked)

☐ Crea tu propia excepción extendiendo Exception (checked) o RuntimeException (unchecked).

```
// Archivo: SaldoInsuficienteException.java
public class SaldoInsuficienteException extends Exception {
    private double cantidadFaltante;

    public SaldoInsuficienteException(double cantidadFaltante) {
        super("Saldo insuficiente. Faltan: " + cantidadFaltante + " €");
        this.cantidadFaltante = cantidadFaltante;
    }

    public double getCantidadFaltante() {
        return cantidadFaltante;
    }
}
```

4.7 Clase con excepción personalizada integrada

```
public class CuentaBancaria {
    private String titular;
    private double saldo;

    public CuentaBancaria(String titular, double saldoInicial) {
        if (saldoInicial < 0)
            throw new IllegalArgumentException("El saldo no puede ser negativo");
        this.titular = titular;
        this.saldo = saldoInicial;
    }

    public void ingresar(double cantidad) {
        if (cantidad <= 0)
            throw new IllegalArgumentException("La cantidad debe ser positiva");
    }
}
```

```

        saldo += cantidad;
    }

    public void retirar(double cantidad) throws SaldoInsuficienteException {
        if (cantidad <= 0)
            throw new IllegalArgumentException("La cantidad debe ser positiva");
        if (cantidad > saldo)
            throw new SaldoInsuficienteException(cantidad - saldo);
        saldo -= cantidad;
    }

    public double getSaldo() { return saldo; }

    @Override
    public String toString() {
        return titular + " | Saldo: " + saldo + " €";
    }
}

```

4.8 Usar la clase anterior en main

```

public static void main(String[] args) {
    CuentaBancaria cuenta = new CuentaBancaria("Ana", 1000);

    // Intentar varias operaciones
    double[] cantidades = {200, 900, -50, 300};

    for (double c : cantidades) {
        try {
            cuenta.retirar(c);
            System.out.println("Retirado: " + c);
        } catch (SaldoInsuficienteException e) {
            System.out.println("Sin saldo: " + e.getMessage());
        } catch (IllegalArgumentException e) {
            System.out.println("Dato inválido: " + e.getMessage());
        } finally {
            System.out.println("Saldo actual: " + cuenta.getSaldo());
        }
    }
}

```

4.9 Excepción en acceso a array (ArrayIndexOutOfBoundsException)

```

int[] arr = {1, 2, 3};

try {
    System.out.println(arr[5]); // índice 5 no existe
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Índice fuera de rango: " + e.getMessage());
}

// Lectura segura de entero con Scanner:
Scanner sc = new Scanner(System.in);
int numero = -1;
boolean valido = false;
while (!valido) {
    try {
        System.out.print("Introduce un entero: ");
        numero = sc.nextInt();
        valido = true;
    } catch (InputMismatchException e) {
        System.out.println("No es un entero, inténtalo de nuevo");
    }
}

```

```
        sc.nextLine(); // limpiar el buffer  
    }  
}
```

5. POO — CONCEPTOS Y EJEMPLOS

5.1 ¿Por qué los atributos deben ser privados?

Los atributos privados protegen el estado interno del objeto. Solo se accede a ellos a través de getters/setters, que pueden incluir validaciones. Así evitamos que desde fuera se pongan valores incoherentes.

```
// SIN encapsulamiento (MAL)
persona.edad = -5; // nadie lo impide

// CON encapsulamiento (BIEN)
persona.setEdad(-5); // el setter valida y no lo permite
```

5.2 Clase con atributo static (contador de objetos)

```
public class Producto {
    private static int contadorProductos = 0;
    private int id;
    private String nombre;
    private double precio;

    public Producto(String nombre, double precio) {
        contadorProductos++;
        this.id = contadorProductos;
        this.nombre = nombre;
        this.precio = precio;
    }

    public static int getContadorProductos() {
        return contadorProductos;
    }

    @Override
    public String toString() {
        return "[" + id + "] " + nombre + " -> " + precio + "€";
    }
}
```

5.3 Clase con validación en constructor y setters

```
public class Alumno {
    private String nombre;
    private double nota;

    public Alumno(String nombre, double nota) {
        if (nombre == null || nombre.isEmpty())
            throw new IllegalArgumentException("Nombre no puede estar vacío");
        if (nota < 0 || nota > 10)
            throw new IllegalArgumentException("Nota debe estar entre 0 y 10");
        this.nombre = nombre;
        this.nota = nota;
    }

    public void setNota(double nota) {
        if (nota >= 0 && nota <= 10) this.nota = nota;
    }

    public boolean aprobado() { return nota >= 5; }
```

```

@Override
public String toString() {
    return nombre + " (" + nota + ")" + (aprobado() ? " APROBADO" : " SUSPENSO");
}
}

```

5.4 Clase que contiene otra clase (composición)

□ Un objeto puede tener como atributo otro objeto. Esto se llama composición.

```

public class Direccion {
    private String calle;
    private String ciudad;

    public Direccion(String calle, String ciudad) {
        this.calle = calle;
        this.ciudad = ciudad;
    }

    public String toString() { return calle + ", " + ciudad; }
}

public class Empleado {
    private String nombre;
    private Direccion direccion; // objeto dentro de objeto

    public Empleado(String nombre, Direccion direccion) {
        this.nombre = nombre;
        this.direccion = direccion;
    }

    public String toString() {
        return nombre + " vive en " + direccion;
    }
}

// En main:
Direccion d = new Direccion("Calle Mayor 1", "Madrid");
Empleado e = new Empleado("Ana", d);
System.out.println(e);

```

5.5 Método equals personalizado

```

public class Punto {
    private double x;
    private double y;

    public Punto(double x, double y) {
        this.x = x; this.y = y;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Punto)) return false;
        Punto otro = (Punto) obj;
        return this.x == otro.x && this.y == otro.y;
    }
}

// En main:
Punto p1 = new Punto(1, 2);
Punto p2 = new Punto(1, 2);
System.out.println(p1 == p2); // false (referencias distintas)

```

```
System.out.println(p1.equals(p2)); // true (mismo contenido)
```

5.6 Clase con menú (patrón habitual en examen)

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int opcion;

        do {
            System.out.println("--- MENÚ ---");
            System.out.println("1. Opción A");
            System.out.println("2. Opción B");
            System.out.println("0. Salir");
            System.out.print("Elige: ");
            opcion = sc.nextInt();

            switch (opcion) {
                case 1: System.out.println("Hiciste A"); break;
                case 2: System.out.println("Hiciste B"); break;
                case 0: System.out.println("Adiós"); break;
                default: System.out.println("Opción inválida");
            }
        } while (opcion != 0);
    }
}
```

5.7 Comparable — Ordenar objetos con Arrays.sort

```
import java.util.Arrays;

public class Alumno implements Comparable<Alumno> {
    private String nombre;
    private double nota;

    public Alumno(String nombre, double nota) {
        this.nombre = nombre;
        this.nota = nota;
    }

    @Override
    public int compareTo(Alumno otro) {
        // Orden ascendente por nota
        return Double.compare(this.nota, otro.nota);
        // Descendente: return Double.compare(otro.nota, this.nota);
    }

    @Override
    public String toString() { return nombre + " -> " + nota; }
}

// En main:
Alumno[] alumnos = {
    new Alumno("Luis", 6.0),
    new Alumno("Ana", 9.0),
    new Alumno("Marta", 7.5)
};

Arrays.sort(alumnos);
for (Alumno a : alumnos) System.out.println(a);
```


6. CONCEPTOS TEÓRICOS CLAVE

Tipos de excepciones

Checked (verificadas): heredan de Exception. El compilador obliga a capturarlas o declararlas con throws. Ejemplo: IOException, SQLException.

Unchecked (no verificadas): heredan de RuntimeException. No obliga a capturarlas. Ejemplo: NullPointerException, ArrayIndexOutOfBoundsException, ArithmeticException.

Interfaces y clases abstractas (solo teoría)

- ☐ Interfaz: contrato con métodos sin implementación. Una clase puede implementar varias interfaces (implements). Ejemplo: Comparable, Serializable.
- ☐ Clase abstracta: clase que no se puede instanciar. Puede tener métodos abstractos (sin cuerpo) que las subclases deben implementar (abstract).
- ☐ Herencia: una clase puede extender otra con extends. Hereda todos sus atributos y métodos. Solo se puede extender UNA clase.

this vs static

this → hace referencia al objeto actual. Solo se usa dentro de métodos de instancia. Sirve para distinguir atributos de parámetros con el mismo nombre.

static → pertenece a la clase, no a un objeto concreto. Se accede con NombreClase.metodo(). No puede usar this.

Diferencia entre array y ArrayList

- ☐ Array: tamaño fijo, más eficiente, admite primitivos (int, double...). ArrayList: tamaño dinámico, solo objetos (Integer, Double...), más métodos útiles.